hashlock.

Penetration Testing

Manifest Web Wallet

March 2025

info@hashlock.com.au

Hashlock Pty Ltd

Table of Contents

CAUTION

THIS DOCUMENT IS A SECURITY PENETRATION TEST REPORT AND MAY CONTAIN CONFIDENTIAL INFORMATION. THIS INCLUDES IDENTIFIED VULNERABILITIES AND MALICIOUS CODE THAT COULD BE USED TO COMPROMISE THE PROJECT. THIS DOCUMENT SHOULD ONLY BE FOR INTERNAL USE UNTIL ISSUES ARE RESOLVED. ONCE VULNERABILITIES ARE REMEDIATED, THIS REPORT CAN BE MADE PUBLIC. THE CONTENT OF THIS REPORT IS OWNED BY HASHLOCK PTY LTD FOR THE USE OF THE CLIENT.

Executive Summary

The Manifest Web Wallet team partnered with Hashlock to conduct a security Penetration Test of their Manifest Web Wallet Project code base. Hashlock manually and proactively reviewed the code to ensure the project's team and community that the deployed tools were secure.

Project Context

Lifted Init Wallet is a cutting-edge, decentralized digital wallet crafted for the Web3 era, empowering users to securely manage, transfer, and explore their crypto assets across multiple chains. With a sleek, user-friendly interface and seamless dApp integrations, Lifted Init ensures that your NFTs and tokens are not only safely stored but also readily accessible for innovative DeFi opportunities. Experience true asset sovereignty with robust security protocols and intuitive design—your gateway to unlocking the full potential of the decentralized ecosystem.

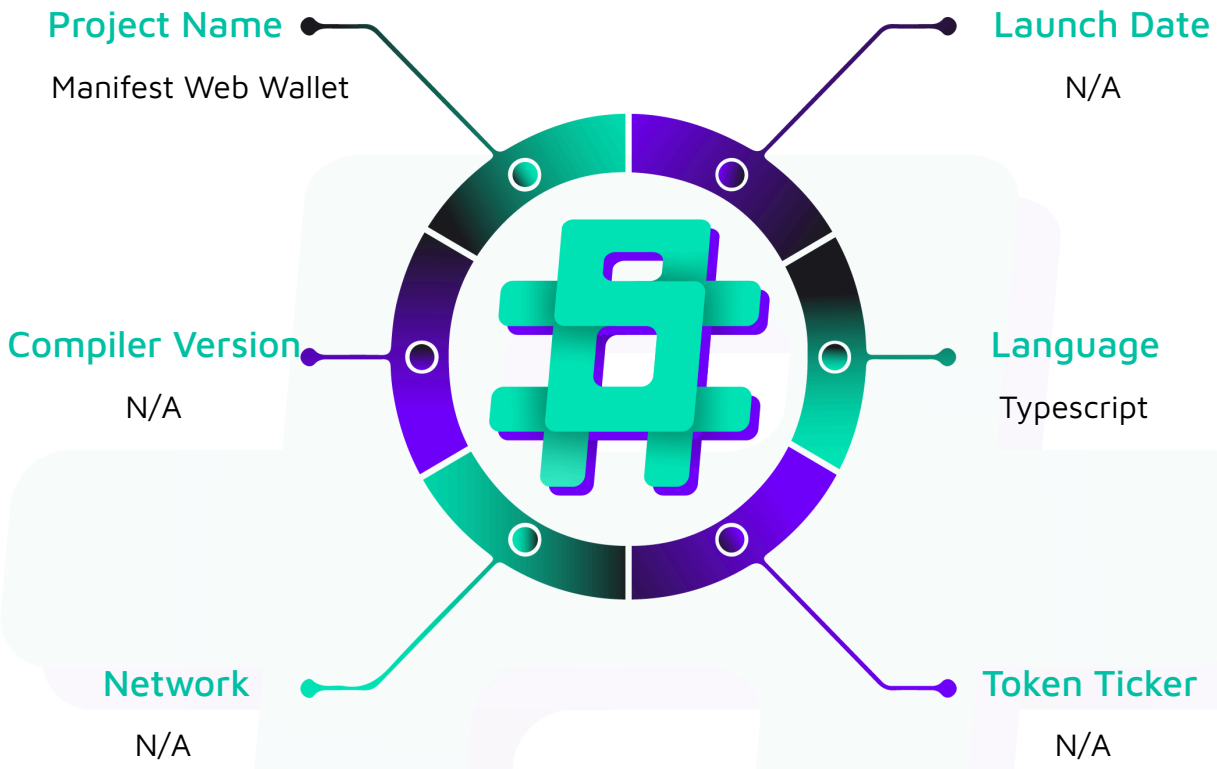
Project Name: Manifest Web Wallet

Compiler Version: N/A

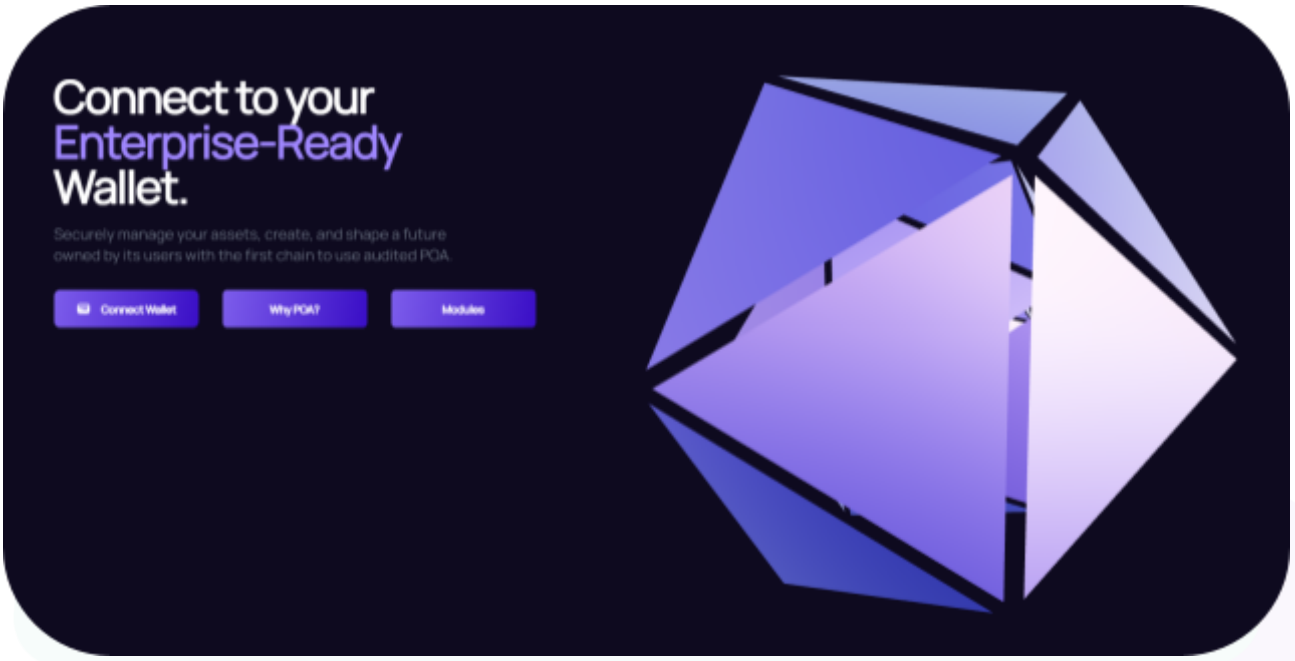
Website: https://wallet.liftedinit.app/

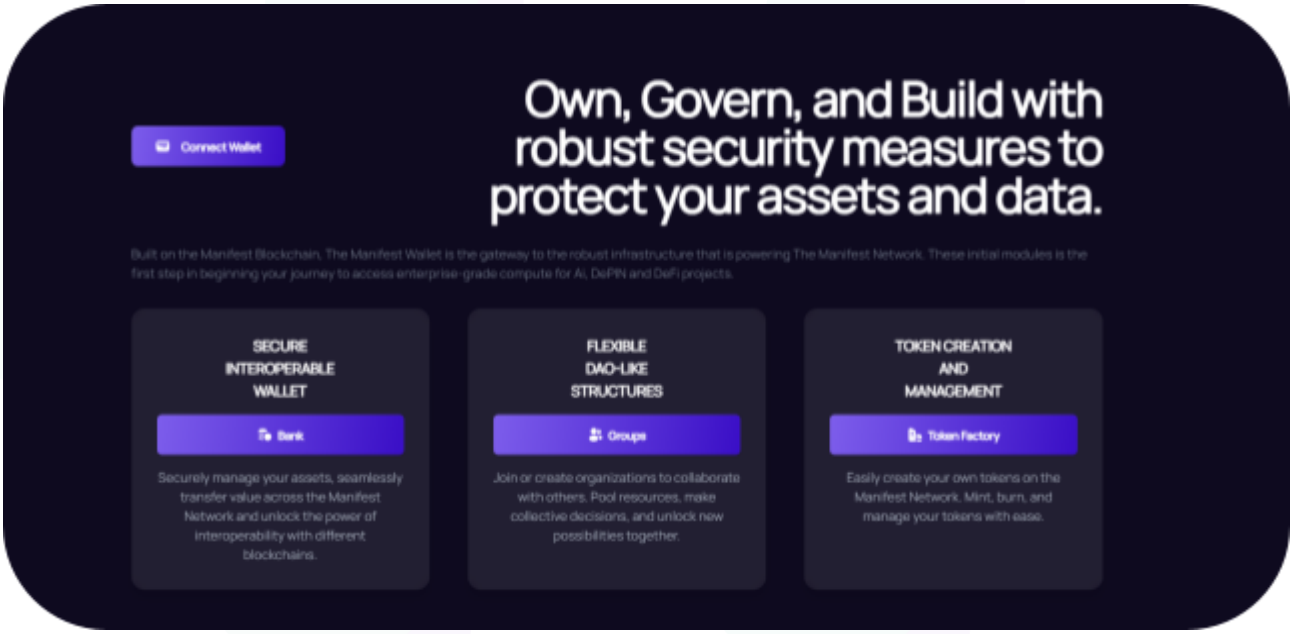
Logo:

Visualised Context:



Project Visuals:





Penetration Test scope

At Hashlock, we executed a comprehensive penetration test on the Manifest Web Wallet project. Our scope included a detailed security assessment, where we rigorously examined the code to identify vulnerabilities and assess overall efficiency. The testing process involved a meticulous manual review, supplemented by advanced software-assisted techniques, to ensure thorough analysis and accuracy.

Description	**Manifest Web Wallet Project code base**
Platform	**Typescript**
Penetration Test Date	**March, 2025**
Repository	https://github.com/liftedinit/manifest-app
GitHub Commit Hash	814c2b268f31035f947b059c6a936faa93941b2f
Fix Review Commit Hash	148b574b7a0081432f5d56cfea47792eaab5fcf2

Note: *Hashlock initially audited the code associated with the "Audited GitHub Commit Hash" and the findings presented in this report pertain specifically to that commit. For the "Fix Review GitHub Commit Hash", Hashlock solely verified the status of the identified findings and did not conduct a comprehensive review to assess any additional code implementations.*

Security Rating

After Hashlock's Penetration Test, we found the code base to be **"Hashlocked"**. The code follows simple logic, with correct and detailed ordering.

The 'Hashlocked' rating is reserved for projects that ensure ongoing security via bug bounty programs or on-chain monitoring technology.

All issues uncovered during automated and manual analysis were meticulously reviewed and applicable vulnerabilities are presented in the Penetration Test Findings section.

We initially identified some vulnerabilities that have since been addressed.

Hashlock found:

1 Medium-severity vulnerabilities

1 Low-severity vulnerabilities

2 QAs

Caution: *Hashlock's Penetration Tests do not guarantee a project's success or ethics, and are not liable or responsible for security. Always conduct independent research about any project before interacting.*

Code Quality

This Penetration Test scope involves the code bases of the Manifest Web Wallet project, as outlined in the Penetration Test Scope section. All contracts, libraries, and interfaces mostly follow standard best practices to help avoid unnecessary complexity that increases the likelihood of exploitation, however, some refactoring was recommended.

The code is very well commented on and closely follows best practice nat-spec styling. All comments are correctly aligned with code functionality.

Penetration Test Resources

We were given the Manifest Web Wallet projects code base code in the form of GitHub access.

As mentioned above, code parts are well-commented. The logic is straightforward, and therefore it is easy to quickly comprehend the programming flow as well as the complex code logic. The comments help understand the overall architecture of the protocol.

Dependencies

Per our observation, the libraries used in this code base's infrastructure are based on well-known industry-standard open-source projects.
Apart from libraries, its functions are used in external code base calls.

Severity Definitions

The severity levels assigned to findings represent a comprehensive evaluation of both their potential impact and the likelihood of occurrence within the system. These categorizations are established based on Hashlock's professional standards and expertise, incorporating both industry best practices and our discretion as security auditors. This ensures a tailored assessment that reflects the specific context and risk profile of each finding.

Significance	Description
High	High-severity vulnerabilities can result in loss of funds, asset loss, access denial, and other critical issues that will result in the direct loss of funds and control by the owners and community.
Medium	Medium-level difficulties should be solved before deployment, but won't result in loss of funds.
Low	Low-level vulnerabilities are areas that lack best practices that may cause small complications in the future.
Gas	Gas Optimisations, issues, and inefficiencies
QA	Quality Assurance (QA) findings are purely informational and don't impact functionality. These notes help clients improve the clarity, maintainability, or overall structure of the code, ensuring a cleaner and more efficient project. They should be addressed for optimization but are not critical to the system’s performance or security.

Status Definitions

Each identified security finding is assigned a status that reflects its current stage of remediation or acknowledgment. The status provides clarity on the handling of the issue and ensures transparency in the auditing process. The statuses are as follows:

Significance	Description
Resolved	The identified vulnerability has been fully mitigated either through the implementation of the recommended solution proposed by Hashlock or through an alternative client-provided solution that demonstrably addresses the issue.
Acknowledged	The client has formally recognized the vulnerability but has chosen not to address it due to the high cost or complexity of remediation. This status is acceptable for medium and low-severity findings after internal review and agreement. However, all high-severity findings must be resolved without exception.
Unresolved	The finding remains neither remediated nor formally acknowledged by the client, leaving the vulnerability unaddressed.

Penetration Test Findings

Medium

[M-01] Liftedinit domains - Open development environments publicly accessible

Description

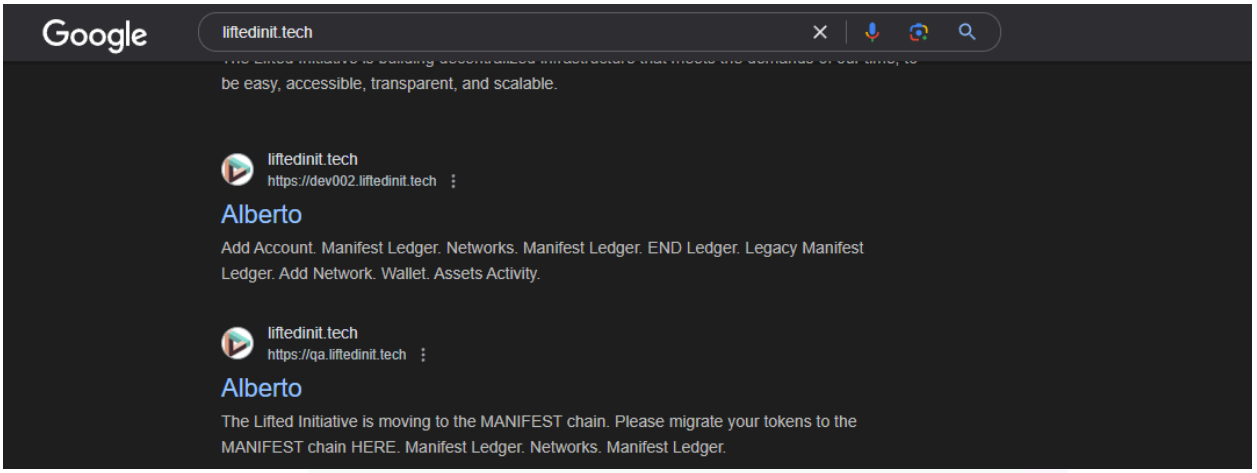
During the security assessment, multiple development environments were discovered to be publicly accessible from the internet and indexed by Google, which makes them easily discoverable.

Vulnerability Details

The security assessment revealed that several development environments are currently accessible without any access controls and are indexed by search engines like Google. These environments can be easily discovered through search queries containing "liftedinit".

While these environments are technically out of scope for the current assessment, they may not be consistently monitored for security issues. There is a risk that these environments could contain information leaks or sensitive data that should not be publicly accessible.

The screenshot below shows an example environment that is findable in that way:



Impact

Since these environments may not be regularly monitored for security issues, potential information leaks or sensitive data exposure might go undetected.

Recommendation

For any exposed public development environments, basic authentication or IP whitelisting should be implemented immediately to prevent open access.

Note

The Manifest team informed Hashlocked this issue is unrelated to the core functionality of the Manifest Network, and while the team recognizes the potential risks, they will continue to track the issue and plan to implement a fix in a future update.

Status

Acknowledged

Low

[L-01] Web wallet - Dependencies not pinned to exact versions

Description

The application uses external dependencies, some of which are not pinned to an exact version but set to a compatible version (^x.x.x). This can potentially allow for dependency attacks. Similar attacks have been seen in the past; for example, Copay's Bitcoin Wallet was attacked in such a way.

Those can be observed in `package.json` and later in `bun.lock` file after building.

Recommendation

Pin all external dependencies to exact versions (x.x.x) instead of using compatible version ranges (^x.x.x) to mitigate the risk of dependency-based attacks.

Status

Resolved

QA

[Q-01] Web wallet - Lack of warning about the possibility of losing contacts or other wallet data

Description

The web wallet application stores contact information and other wallet data in the browser's local storage without clearly communicating this to users. While there is functionality to export contacts, less technically aware users might not understand that clearing browser data will result in the loss of their stored information. This is primarily a user experience concern rather than a security issue.

Recommendation

Add clear messaging to inform users that their data is stored locally and can be lost when clearing browser data.

Status

Resolved

[Q-02] [TODO] manifest-app/pages/admins.tsx - Hardcoded admin address

Description

In the `manifest-app/pages/admins.tsx` file, an admin address is hardcoded as "`manifest1afk9zr2hn2jsac63h4hm60vl9z3e5u69gndzf7c99cqge3vzwjzsfmy9qj`". While this also does not present a security risk since all actions still require blockchain transaction signing for validation, the presence of a hardcoded admin address is unnecessary and may cause unwarranted concerns among users who review the code.

Recommendation

Remove the hardcoded admin address if it serves no functional purpose.

Status

Acknowledged

Conclusion

After Hashlock's analysis, the Manifest Web Wallet project seems to have a sound and well-tested code base now that our vulnerability findings have been resolved and acknowledged. Overall, most of the code is correctly ordered and follows industry best practices. The code is well commented on as well. To the best of our ability, Hashlock is not able to identify any further vulnerabilities.

Our Methodology

Hashlock strives to maintain a transparent working process and to make our Penetration Tests a collaborative effort. The objective of our security Penetration Tests is to improve the quality of systems and upcoming projects we review and to aim for sufficient remediation to help protect users and project leaders. Below is the methodology we use in our security Penetration Test process.

Manual Code Review:

In manually analysing all of the code, we seek to find any potential issues with code logic, error handling, protocol and header parsing, cryptographic errors, and random number generators. We also watch for areas where more defensive programming could reduce the risk of future mistakes and speed up future Penetration Tests. Although our primary focus is on the in-scope code, we examine dependency code and behavior when it is relevant to a particular line of investigation.

Vulnerability Analysis:

Our methodologies include manual code analysis, user interface interaction, and white box penetration testing. We consider the project's website, specifications, and whitepaper (if available) to attain a high-level understanding of what functionality the code base under review contains. We then communicate with the developers and founders to gain insight into their vision for the project. We install and deploy the relevant software, exploring the user interactions and roles. While we do this, we brainstorm threat models and attack surfaces. We read design documentation, review other Penetration Test results, search for similar projects, examine source code dependencies, skim open issue tickets, and generally investigate details other than the implementation.

Documenting Results:

We undergo a robust, transparent process for analysing potential security vulnerabilities and seeing them through to successful remediation. When a potential issue is discovered, we immediately create an issue entry for it in this document, even though we still need to verify the feasibility and impact of the issue. This process is vast because we document our suspicions early even if they are later shown not to represent exploitable vulnerabilities. We generally follow a process of first documenting the suspicion with unresolved questions, and then confirming the issue through code analysis, live experimentation, or automated tests. Code analysis is the most tentative, and we strive to provide test code, log captures, or screenshots demonstrating our confirmation. After this, we analyse the feasibility of an attack in a live system.

Suggested Solutions:

We search for immediate mitigations that live deployments can take and finally, we suggest the requirements for remediation engineering for future releases. The mitigation and remediation recommendations should be scrutinised by the developers and deployment engineers, and successful mitigation and remediation is an ongoing collaborative process after we deliver our report, and before the contract details are made public.

Disclaimers

Hashlock's Disclaimer

Hashlock's team has analysed the code bases in accordance with the best industry practices at the date of this report, in relation to: cybersecurity vulnerabilities and issues in the code base source code, the details of which are disclosed in this report, (Source Code); the Source Code compilation, deployment, and functionality (performing the intended functions).

Due to the fact that the total number of test cases is unlimited, the Penetration Test makes no statements or warranties on the security of the code. It also cannot be considered a sufficient assessment regarding the utility and safety of the code, bug-free status, or any other statements of the contract. While we have done our best in conducting the analysis and producing this report, it is important to note that you should not rely on this report only. We also suggest conducting a bug bounty program to confirm the high level of security of this code base.

Hashlock is not responsible for the safety of any funds and is not in any way liable for the security of the project.

Technical Disclaimer

Code is deployed and executed on a platform. The platform, its programming language, and other software related to the code base can have their own vulnerabilities that can lead to attacks. Thus, the Penetration Test can't guarantee the explicit security of the Penetration Tested code bases.

About Hashlock

Hashlock is an Australian-based company aiming to help facilitate the successful widespread adoption of distributed ledger technology. Our key services all have a focus on security, as well as projects that focus on streamlined adoption in the business sector.

Hashlock is excited to continue to grow its partnerships with developers and other web3-oriented companies to collaborate on secure innovation, helping businesses and decentralised entities alike.

Website: hashlock.com.au
Contact: info@hashlock.com.au